



Accessibility Testing

? What you'll learn

- Strategies for ensuring your application works for users with disabilities
- What accessibility scans are available via plugins vs [Cypress Accessibility](#)
- Performance impacts of in-test accessibility checks
- How to write accessibility-specific tests
- The difference between Testing Library locators and Test IDs

Accessibility testing helps to confirm that an application works correctly for people with disabilities.

To set a good foundation for an accessible user experience, web sites and apps must conform to certain guidelines, known as the [Web Content Accessibility Guidelines](#) (WCAG). Meeting or exceeding these guidelines will help ensure your disabled users can independently perceive the content of your application, navigate through your pages and sections, and complete the available actions.

Cypress supports many kinds of accessibility testing, through standard test practices, a rich plugin ecosystem, as well as [Cypress Accessibility](#), a commercial, enterprise-ready solution in Cypress Cloud.

Including accessibility in your Cypress tests

Here are the main ways to account for accessibility when testing with Cypress:

Accessibility scans

- [In-test plugins](#)



- [Cypress Accessibility](#)

Cypress Accessibility is a paid premium solution. [Schedule a demo](#) today and see how easy it is to enhance your accessibility testing while speeding up your development process.

Explicit, application-specific tests

- Accessibility-focused assertions and locators
- Tests for specific functionality relevant to assistive technology

We will discuss each of these below.

Accessibility Scans

Automated scans are effective because many accessibility issues are caused by incorrect HTML structure or missing data that would be required by assistive technologies like screen readers, voice control systems, Braille displays, and more. Browsers transform the current HTML on the page into a standardized format that other technologies can hook into. Automated scans in your tests will alert you if specific items needed for this process are missing.

Note that while automated scans do a good job at detecting violations of a known list of rules, no automated scan can prove that the interface is **fully** accessible and works well for users with disabilities. It is always necessary to understand the limitations and expected coverage provided by the library you choose, and then make a plan to cover the gaps with manual testing and/or traditional Cypress assertions about additional expected behavior, based on your knowledge of what is needed by your users.

In-test plugins



The `cypress-axe` plugin is a community plugin that integrates the popular [Axe Core® library by Deque Systems](#) into your Cypress tests.

After setting up the plugin, commands are added to your project to inject the library and use it to run an accessibility check. Running the included `checkA11y()` command performs a scan of the current state of the page or component that you are testing, and you can choose to fail the test in response to accessibility issues detected. Detailed configuration is available to scope this to the specific WCAG Success Criteria and related rules that you want to test.

There are also some newer community plugins that are themselves built on top of `cypress-axe` and extend it in various ways, such as `wick-a11y` and `cypress-a11y-report`, as well as other accessibility testing libraries like the [IBM Equal Access Checker](#). All of these tools have their own dedicated documentation for installation, setup, and executing the checks, which we won't repeat here.

The general "add a command to trigger a scan" approach helps detect and prevent all-too-common issues like poor color contrast, missing labels for icons and buttons, images without alt text, and other errors in the implementation of a user interface that can be detected with generic checks.

Managing test performance

In-test automated accessibility checks are so valuable because they bundle a large amount of assertions about all kinds of rules and criteria into one simple command. But this also means they come with some overhead: for example, each call to `cy.checkA11y()` evaluates DOM elements to see which accessibility rules apply to each element in the page, and then runs specific DOM checks for those elements for each of the rules, and computes the results, all of which takes time to complete. If other assets like screenshots are generated at the same time, the performance of this should also be considered, and assets should be generated only when needed.

Generally this is not a problem if you have a small number of accessibility checks, but it can become a big contributor to your pipeline's overall execution time as you scale up your accessibility testing and want to run checks against hundreds or even thousands of unique states of complex applications and workflows. Since accessibility checks tend to be added gradually over time, the time taken is often not obvious, but it is definitely worth paying attention to.



To avoid performance bloat, you will want to actively manage the tradeoff between test performance and how often tests are executed. This is sometimes solved by testing only the initial pageload, or the state at the end of a test, but this approach misses many important states like modals, multistep workflows, error states in forms, open states of menus, and more. And since many tests may begin or end at the same page state as other tests, there can be a lot of redundant scans, so it pays to be more precise and intentional in what you test.

Performance can also be managed by targeting the accessibility checks to only certain rules or certain parts of the page, or confining them to component tests. Testers should be trained on how and when to run accessibility checks to get the best feedback possible while eliminating redundant checks that don't add value, and minimizing the disruption to the test pipeline.

It is best to proactively manage this, because unnecessary flake, failures, or timeouts related to accessibility checks can cause frustration and lead to resistance in adopting this form of testing, which in turn makes it harder to gain momentum towards meeting your accessibility goals.

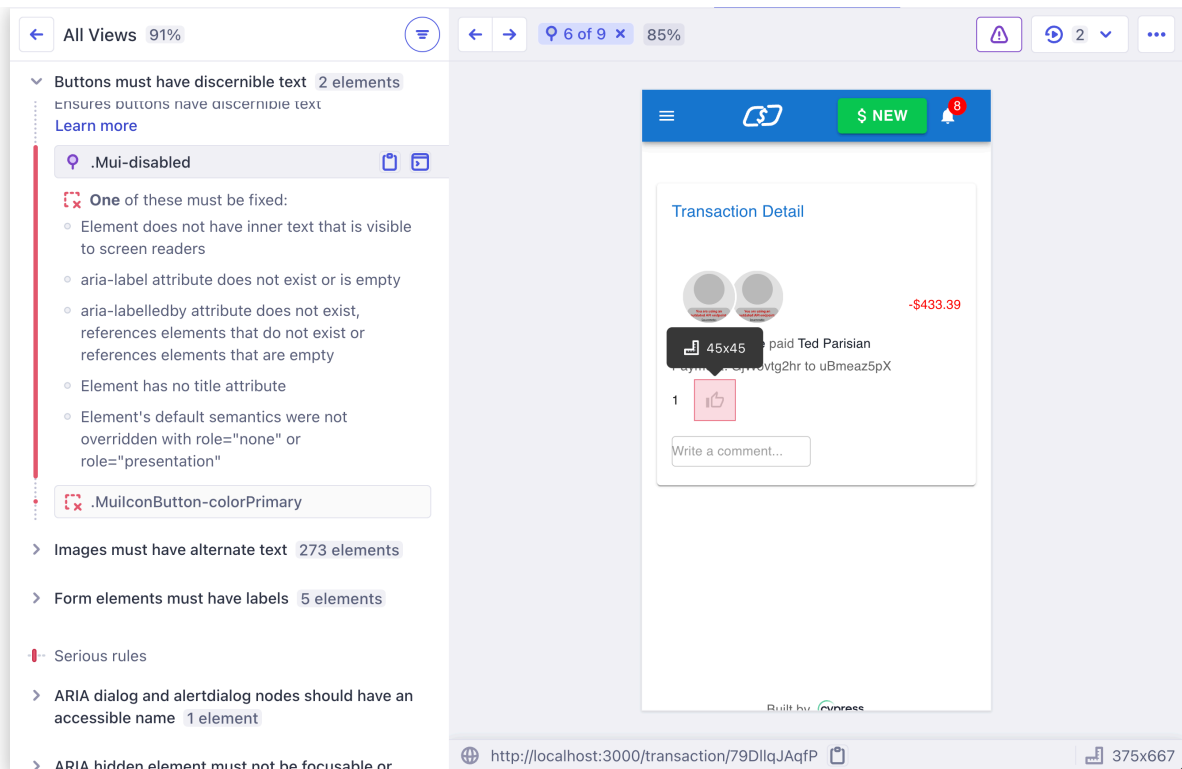
Cypress Accessibility

In-test accessibility checks are the only kind available in typical testing scenarios. They come with some limitations and tradeoffs that [Cypress Accessibility](#), available in Cypress Cloud, is designed to solve. By moving the checks outside of the test context and running them in Cypress Cloud as tests are recorded, Cypress Accessibility removes adoption, training, and test performance hurdles that can hinder the effective implementation of in-test checks. Cypress automatically detects all the steps within user flows, requiring no test code to be written.

To learn more, you can read our [dedicated docs](#), or review a [public live example of an automatically-generated accessibility report](#) in our Cypress Realworld App demo project.

Cypress Accessibility is a paid premium solution. [Schedule a demo](#) today and see how easy it is to enhance your accessibility testing while speeding up your development process.





Accessibility-specific tests

While automation like Axe Core® can detect missing attributes and other aspects of code quality that impact the experience of people with disabilities using the web, this kind of automation doesn't know anything about your specific application and the expectations you have for your users. That's where including accessibility in your specs comes in.

Keyboard navigation

To test in-page navigation with the keyboard, you can use the `cy.press()` to dispatch native tab events. This can also be used to test custom behavior like autocomplete.

Asserting alt text of images

To assert that the correct alternative text is present on your logo image, an explicit test can be written:

```
it('adds todos', () => {  
  cy.visit('https://example.cypress.io/')  
  // explicitly check the alt text of an image
```



```
cy.get('img[data-cy="logo"]').should('have.attr', 'alt',  
'Cypress Logo')  
})
```

It's also possible to use an accessibility-aware locator approach to find the element while performing some other assertion. Here instead of using the element's test ID, we locate the image by its `alt` text. This is more concise, and the decision of which format to prefer is up to your team's preferences:

```
// use the `alt` content to target the image  
cy.get('img[alt="Cypress Logo"]').should('be.visible')
```

This kind of accessibility-friendly locator approach is also possible with the [Cypress Testing Library](#) plugin, which provides some helpers for this purpose:

```
// use the recommended ByRole Testing Library locator  
cy.findByRole('img', { name: 'Cypress Logo'  
}).should('be.visible')
```

Asserting the accessible name of a button

A similar accessibility-minded locator technique can be used with interactive elements like buttons:

```
// click the "Submit" button located by `cy.contains()`  
cy.contains('button', 'Submit').click()
```

And using the Testing Library `ByRole` locator:

```
// click any element with role `button` and an accessible name  
of `Submit`  
cy.findByRole('button', { name: 'Submit' }).click()
```

There are some important differences to consider between locating specific HTML elements themselves and locating those same elements only by their `role`,



which is the main Testing Library recommendation. We'll discuss these in the next section.

Test IDs, Testing Library, and Accessibility

The use of `data-cy`-style test ID attributes to help with test stability has been a longstanding [best practice recommendation](#) by Cypress. Test IDs are resilient to non-functional UI changes because they don't specify anything about the nature of the code or content itself, only that some element with that data attribute is present. When code changes, as long as the `data-cy` attributes are preserved in the right places, all the tests using them should continue to pass.

This approach explicitly avoids testing accessibility - which is all about the nature and structure of the content being tested, and how well the implementation in code matches that content, to provide a functional experience in a range of assistive technologies and browser setups.

Testing Library (introduced in 2018 and now widely popular) is sometimes seen as a solution to the fact that explicit Test IDs do not provide any accessibility testing benefit. While Testing Library's helpers can be convenient and useful in their own right, and they keep accessibility top-of-mind for testers, it's important to note that Testing Library locators do **not** guarantee the accessibility of the elements being tested.

Whether you primarily use Testing Library element locators, test IDs, or a different approach, testing for accessibility requires making specific assertions about the **implementation and behavior** of your application.

There is much more to this topic than we can cover here, but we will provide a small example, and a recommendation for what to do.

Asserting about roles vs elements – button example

The native HTML `button` element has a complex accessibility contract implemented by the browser for keyboard and screen reader users. Here is a partial list of ways a button element is automatically managed by the browser:



- It has the implicit `role` of `button` so that users know what it is - a screen reader, for example, would announce it as a button
- It has a default browser button style to visually distinguish it from surrounding content
- It has default browser styles for hover and "pressed" states
- It is placed in the tab order of the document and can be focused with the keyboard
- When focused it receives a focus outline style
- It can be activated with a mouse click, or the enter key, or the space bar
- If placed inside a `form` element, activating the button by any of these methods will submit the form (because its default type is `submit`)
- It will receive specific styling, customizable by the user, in Windows High Contrast Mode (used by people with certain visual disabilities)

On the other hand, a `div` with `role` of `button` does not get any default browser accessibility behavior, and is **not accessible** without custom JavaScript and CSS replacing the browser contract.

This means that, without some assertions about either the `button` element itself, or a full test of the accessibility "contract" of that element, it is not safe for a developer to refactor from a `button` to a `<div role="button">`, even though Testing Library will treat each as the same if located using the `ByRole` locator. This is because, much like test IDs, the `ByRole` approach is intended to *avoid* testing the implementation directly, to keep tests resilient when code is refactored.

Developers who are unfamiliar with accessibility may assume that if a Testing Library `ByRole` locator can be made to pass before and after a code change, there has been no functional or accessibility-related change in the underlying element. As we've seen, this is not actually the case, because of the extra behavior browsers only implement for native HTML elements. For more about this difference and why semantic HTML elements are preferred, see the [first rule of Accessible Rich Internet Applications \(ARIA\)](#).

Where to test accessibility

So what should you do in your test automation to help confirm the UI is accessible? First of all, for known critical areas like forms or checkout flows,



ensure that the accessibility behavior is tested explicitly in at least one place. The means verifying that form fields and buttons have the correct labels and use the expected HTML elements, and other aspects of the DOM that communicate necessary information.

Component tests are a fantastic place to do this centralized accessibility spec work, whether using [Cypress Component testing](#) or something else. Component tests are run locally during development, written by developers as they build the UI, and can easily specify the specific HTML that is intended to be rendered, without being a distraction in an end-to-end test of a specific user flow.

Accessibility should be tested at least once for a given component, area of the application, or workflow. Once that is achieved, there is no additional accessibility benefit to repeatedly asserting subsets of the same things in other tests, unless those tests introduce new combinations of components that have not been tested for accessibility anywhere else.

Choosing a locator approach

Test IDs are the most resilient locator option and are a good choice for maximizing the stability of your pipeline, especially if end-to-end tests are written by people who are not familiar with accessibility, and don't already know the correct roles, elements, and behavior that should be expected. You do not lose any accessibility "coverage", as long as accessibility is explicitly tested with assertions, and a library like Axe Core® is in place to catch mistakes.

Testing Library locators are convenient and familiar to many React developers (due to its origins as "React Testing Library"), and they don't require any code changes, but may cause *many* tests to fail when a label is missing, instead of just your explicit accessibility assertions, because they are a step closer to the implementation details.

Additionally, Testing Library locators can provide a false sense of confidence that something is accessible simply because it can be located by its role. Still, when understood correctly, teams can write effective tests using this approach, and some locators like `findByLabelText` stand out as being particularly useful.

What is most important in all accessibility automation discussions is an this: how can your testing strategy best support your users with disabilities to independently understand and use your application? Cypress enables you to place the user at the center of your testing process. With a mix of automated scans and some specific intentional assertions, you can reach high levels of



confidence that your application is free of known accessibility errors and let your test pipeline warn you if things are going off track.

See also

- [Cypress Accessibility](#)
- [Plugins](#)

Last updated on **Apr 30, 2026**

CONTENTS

